

Specification: Static Local RAG + Hybrid Retrieval + Cross-Encoder Re-Ranking + R2 Conversation Memory

1. Overview

This specification defines a fully serverless Retrieval-Augmented Generation (RAG) pipeline for the Ragtime/BrokerAI website. The system runs entirely inside Vercel serverless functions and uses:

- **Static local RAG artifacts** (embeddings, BM25 index, cross-encoder model)
- **Hybrid retrieval** (dense + sparse)
- **Cross-encoder re-ranking** (local ONNX model)
- **Conversation history stored in Cloudflare R2**
- **Azure OpenAI** for final answer generation

No vector database, no Python runtime at inference, and no persistent server processes are required.

2. Corpus Chunking

2.1 Chunk Size

- Target: **200–300 tokens**
- Maximum: **350 tokens**

2.2 Overlap

- **50–100 token overlap** between consecutive chunks

2.3 Chunk Boundaries

Split on semantic boundaries:

- Headings
- Paragraphs
- Bullet lists
- Code blocks

2.4 Normalization

- Remove excessive whitespace
- Preserve headings and section markers

2.5 Output Format (chunks.json)

```
json
[
  {
    "id": "chunk_0001",
    "doc_id": "roadmap_v1",
    "section": "Meta-agent architecture overview",
    "text": "Ragtime's meta-agent architecture consists of...",
    "tokens": 245
  }
]
```

3. Offline Build Step

Performed once during build.

3.1 Dense Embeddings (embeddings.json)

- Model: **all-MiniLM-L6-v2** (384-dim)
- For each chunk:
 - Compute embedding
 - Store as float array

Format:

```
json
[
  {
    "id": "chunk_0001",
    "embedding": [0.0123, -0.4421, 0.9982, ...]
  }
]
```

3.2 Sparse BM25 Index (bm25.json)

- Use Python `rank_bm25` or equivalent
- Tokenize chunks (lowercase, whitespace split)

- Compute:
 - tokenized docs
 - IDF values
 - average document length

Format:

json

```
{
  "doc_ids": ["chunk_0001", "chunk_0002"],
  "tokenized_docs": [
    ["ragtime", "meta-agent", "architecture"],
    ["resolver", "legacy", "erp"]
  ],
  "idf": { "ragtime": 3.2, "meta-agent": 4.1 },
  "avgdl": 220.5
}
```

3.3 Cross-Encoder Model

- Model: **ms-marco-MiniLM-L-6-v2-cross-encoder** (or equivalent)
- Export ONNX format compatible with @xenova/transformers
- Store under: models/cross_encoder/

4. Runtime Retrieval Pipeline (Vercel Serverless)

4.1 Inputs

- User query
- Session ID (cookie)
- Conversation history (from R2)

4.2 Dense Retrieval

- Load embeddings.json at module scope
- Convert embeddings to Float32Array
- Compute query embedding using @xenova/transformers

- Compute cosine similarity for all chunks

4.3 Sparse Retrieval (BM25)

- Load bm25.json at module scope
- Tokenize query
- Compute BM25 score for each chunk

4.4 Hybrid Scoring

For each chunk:

$\text{hybrid_score} = \alpha * \text{dense_score} + (1 - \alpha) * \text{sparse_score}$

- Default $\alpha = 0.5$
- Select **top-N = 20** candidates

4.5 Cross-Encoder Re-Ranking

For each of the top-N candidates:

Input:

- (query, chunk.text)

Output:

- relevance score (float)

Sort by cross-encoder score.

Select **top-k = 10** final chunks.

5. Conversation Memory (Cloudflare R2)

5.1 Storage Model

Bucket: ragtime-conversations

Key format: conversations/{sessionId}.json

5.2 Session ID Generation

- On first request without cookie:
 - Generate sessionId = crypto.randomUUID()
 - Set cookie rag_session
- Store session ID in R2

5.3 Fingerprint (Optional Heuristic)

Compute: fingerprint = sha256(ip + userAgent)

Store in session file for possible continuity detection.

5.4 Returning User Logic

If no session cookie:

1. Compute fingerprint
2. Search R2 for matching fingerprint
3. If found:
 - Ask user: "A previous conversation may belong to you. Continue it?"
4. If yes → load previous session
5. If no → create new session

5.5 Conversation File Format

json

```
{
  "sessionId": "uuid",
  "fingerprint": "hashed-ip-ua",
  "history": [
    { "role": "user", "content": "Explain Ragtime architecture." },
    { "role": "assistant", "content": "Ragtime is a meta-agent framework..." }
  ],
  "summary": "User is exploring Ragtime meta-agent and resolver design.",
  "lastSeen": "2026-08-13T15:18:00Z"
}
```

5.6 History Management

- Append new messages each turn
- If history grows too large:
 - Summarize using Azure OpenAI
 - Truncate older messages
 - Keep summary + last few turns

6. Final Answer Generation (Azure OpenAI)

6.1 Prompt Composition

Include:

1. System instructions
2. Conversation summary
3. Recent conversation turns
4. Top-k retrieved chunks
5. User query

6.2 Flow

1. Retrieve → hybrid → cross-encoder re-rank → top-k chunks
2. Load conversation history from R2
3. Build final prompt
4. Call Azure OpenAI
5. Return answer
6. Append assistant message to R2 session file

7. Performance Expectations

- Dense + sparse + hybrid: **<10 ms**
- Cross-encoder re-ranking (20 pairs): **20–40 ms**
- Azure OpenAI answer: **200–400 ms**
- Total: **~250–450 ms** typical

Corpus size: **~3,000 chunks** Embedding dim: **384** All operations fit comfortably within Vercel serverless limits.

8. Deliverables for Claude

Claude should produce:

1. **Chunking script**
2. **Embedding generation script**
3. **BM25 index builder**
4. **Cross-encoder ONNX export + loader**

5. **Hybrid retrieval implementation**
6. **Cross-encoder re-ranking implementation**
7. **R2 session management utilities**
8. **Next.js API route** integrating all components
9. **Prompt templates** for:
 - summarization
 - re-ranking (if needed)
 - final answer generation

9. Repository Folder Structure (Static RAG + Hybrid + Cross-Encoder + R2 Memory)

```

/
├─ src/
│   ├─ api/
│   │   └─ rag/
│   │       ├─ retrieve.ts # hybrid retrieval + cross-encoder re-ranking
│   │       ├─ answer.ts  # final Azure OpenAI answer generation
│   │       ├─ session.ts  # session ID + fingerprint + R2 persistence
│   │       └─ index.ts    # main entrypoint for chat API route
│   │   └─ health.ts      # optional health check
│   │
│   └─ rag/
│       └─ loaders/
│           ├─ loadEmbeddings.ts # loads embeddings.json into Float32Array
│           ├─ loadBM25.ts       # loads bm25.json
│           ├─ loadChunks.ts     # loads chunks.json
│           └─ loadCrossEncoder.ts # loads ONNX cross-encoder model
│       │
│       └─ retrieval/
│           ├─ dense.ts          # cosine similarity
│           ├─ sparse.ts         # BM25 scoring
│           └─ hybrid.ts         # hybrid scoring (dense + sparse)

```

```

|   |   |   └─ rerank.ts           # cross-encoder re-ranking
|   |   |
|   |   └─ utils/
|   |   |   └─ tokenize.ts         # tokenizer for BM25 + query
|   |   |   └─ normalize.ts        # text normalization
|   |   |   └─ scoring.ts          # score combination helpers
|   |   |   └─ types.ts            # shared TypeScript types
|   |   |
|   |   └─ prompts/
|   |       └─ answerPrompt.ts      # final answer generation prompt
|   |       └─ summaryPrompt.ts     # conversation summarization prompt
|   |       └─ rerankPrompt.ts      # optional LLM-based re-ranking prompt
|   |
|   └─ r2/
|       └─ client.ts                # R2 SDK client
|       └─ readSession.ts           # load conversation from R2
|       └─ writeSession.ts          # save conversation to R2
|       └─ findByFingerprint.ts     # optional fingerprint-based lookup
|
└─ rag_data/
    └─ chunks.json                  # chunked corpus (200-300 tokens, overlap)
    └─ embeddings.json              # dense embeddings (MiniLM-L6-v2)
    └─ bm25.json                    # sparse BM25 index
    └─ metadata.json                # optional corpus metadata
    |
└─ models/
    └─ cross_encoder/
        └─ model.onnx               # ONNX cross-encoder model
        └─ config.json              # model config
        └─ tokenizer.json           # tokenizer for cross-encoder
    |
└─ build/

```



```

|   └─ chunk/
|   |   └─ chunk_corpus.py          # chunking script (Python)
|   └─ embed/
|   |   └─ generate_embeddings.py    # embedding generation (Python or Node)
|   └─ bm25/
|   |   └─ build_bm25.py            # BM25 index builder
|   └─ export/
|       └─ export_cross_encoder.py  # ONNX export script
|
└─ scripts/
    └─ validate_rag.ts              # checks consistency of JSON files
    └─ benchmark_retrieval.ts       # local performance tests
    └─ test_reranker.ts             # cross-encoder test harness
|
└─ public/
    └─ ui_assets/                  # optional UI assets
|
└─ package.json
└─ tsconfig.json
└─ vercel.json
└─ README.md

```

Notes for Claude (implementation guidance)

Build-time artifacts

Everything under `/rag_data` and `/models` is generated offline and committed to the repo.

Runtime

Only `/src/api` and `/src/rag` are executed inside Vercel serverless.

R2 integration

All conversation history logic lives under `/src/r2`.

Cross-encoder

Loaded once per cold start via `loadCrossEncoder.ts`.

Hybrid retrieval

Dense + sparse + hybrid + cross-encoder re-ranking is implemented under `/src/rag/retrieval`.